

Relatório de Auditoria de Segurança

— CityNews Cuiabá

Data: 04/10/2026 · Branch: claude/saas-security-audit-v2-p5kwh9 · Commit: cdee487

Escopo auditado

Código da aplicação em src/ (Next.js App Router): 33 arquivos route.ts (37 handlers) e 25 arquivos "use server" (141 Server Actions + 1 ação inline), todos percorridos.

Banco: as 58 migrations de supabase/migrations/ (cerca de 73 tabelas, views, cerca de 75 funções security definier, Storage, grants), supabase/config.toml, supabase/seed.sql e supabase/bootstrap/.

Configuração e entrega: .env.example, .gitignore, .github/workflows/*, scripts/, next.config.ts, CSP (src/lib/security/headers.ts, src/proxy.ts), service worker (src/sw/).

Histórico git completo (git log --all, 213 commits), procurando JWTs, chaves sk-, ghp_, AKIA, chaves privadas, .env e logs commitados.

Fora do escopo: o ambiente de produção (Vercel/Supabase) não foi consultado. Achados que dependem de configuração indicam a condição.

Stack detectada

TypeScript strict · Next.js (App Router, React 19, Server Components e Server Actions) · Supabase (Postgres com RLS, Auth/GoTrue, Storage, pgmq, pg_cron, pg_net, Vault) acessado por @supabase/ssr/supabase-js (cliente de sessão com RLS e cliente service role só no servidor) · autorização por papéis em user_roles com escopo por editoria (src/lib/auth/permissions.ts, require-role.ts, studioAction) espelhada no banco por funções has_role/can_edit_section · cron/worker com Authorization: Bearer CRON_SECRET · IA via OpenRouter · Vercel + GitHub Actions (CI, Lighthouse, backup). Modelo de isolamento: não é multi-tenant; o isolamento é entre (a) público anônimo, (b) leitor com conta (dono dos próprios dados) e (c) equipe com papéis e editorias.

Nota metodológica

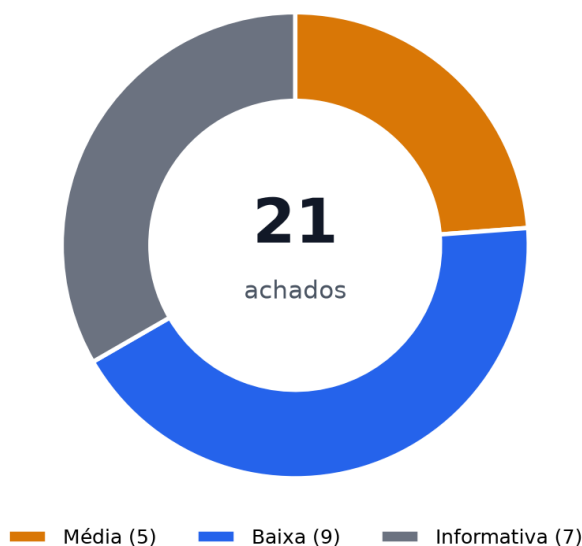
Categoria	Como foi mapeada para esta stack
1. Banco sem tranca (isolamento por RLS)	O projeto usa Supabase e o navegador fala direto com o PostgREST usando a anon key, então o isolamento é a RLS. Inventariamos cada create table e confirmamos enable row level security. Lemos cada política (inclusive as recriadas depois), as views (security_invoker ou não), os grants para anon/authenticated, as políticas de Storage e as cerca de 75 funções security definer: search_path, revoke/grant e checagem de auth.uid(). Também verificamos onde a service role é usada em src/.
2. Permissão definida no navegador	Server Actions são endpoints POST chamáveis diretamente, então esconder um botão por papel não protege nada. Cruzamos cada gate de papel da UI (canEdit, canPublish, canApprove, canRequestUrgent...) com a Server Action ou rota correspondente e com a checagem no banco (RPC ou trigger). Também procuramos validação que dependa de um valor vindo do próprio navegador.
3. IDOR (objeto por ID sem checar posse ou escopo)	Percorremos todos os 37 handlers de rota e as 142 ações. Para cada um que recebe um ID (path, query, form ou corpo), verificamos se a posse ou o escopo (editoria, autoria, token do dono) é checado sobre o objeto, e se a consulta usa o cliente da sessão (RLS aplicada) ou a service role (RLS ignorada).
4. Chaves expostas (hardcode)	Busca por padrões de segredo (JWT eyJhbGci, sk-, sk-or-, ghp_, github_pat_, AKIA, sb_secret_, BEGIN PRIVATE KEY, VAPID, senhas) em arquivos rastreados, workflows, scripts, migrations (chamadas de pg_cron/pg_net), config.toml, seed e docs. O mesmo no histórico git com git log --all -G. Verificamos fallbacks ?? "default" que viram segredo real, validação de startup e quais variáveis NEXT_PUBLIC_* entram no bundle.
5. Inputs sem tratamento (XSS e injeções de saída)	Busca por dangerouslySetInnerHTML, innerHTML, insertAdjacentHTML, document.write, eval e new Function. Verificamos renderizadores de markdown, href/src vindos de dados externos (javascript:), JSON-LD em <script>, templates de e-mail, geração de ICS e CSV, uploads (SVG), o clique de notificação no service worker e a aplicação de sanitizeExternalText. Avaliamos a CSP como mitigação.

Resumo executivo

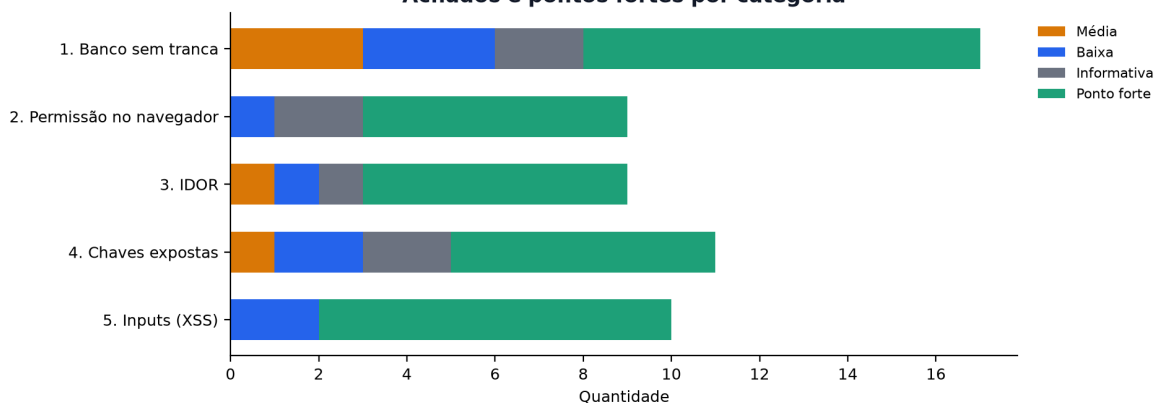
A base tem defesa em profundidade madura: RLS ligada em 100% das tabelas (também nas partições), Server Actions do Estúdio passando por um wrapper único que revalida papel e escopo do objeto, regra de duas pessoas imposta por trigger no banco, CSP com nonce e strict-dynamic, JSON-LD escapado, segredo de cron comparado em tempo constante e nenhum segredo real no código nem no histórico git. Não encontramos achado crítico nem alto. Os 5 achados médios estão no banco (versões de matérias legíveis por anônimos, função de auditoria aberta a qualquer conta, exportação por e-mail sem confirmação), em um escopo de editoria que não é respeitado na remoção em lote de imagens e no .gitignore, que não cobre .env.prod. Nesta stack todo visitante pode criar conta (enable_signup = true), então "qualquer usuário autenticado" equivale a "qualquer pessoa na internet".

Crítica	Alta	Média	Baixa	Informativa	Pontos fortes
0	0	5	9	7	35

Achados por severidade



Achados e pontos fortes por categoria



Pontos fortes

O que foi verificado e está protegido, com evidência no código. Esta seção também documenta a cobertura da auditoria.

1. Banco sem tranca (isolamento por RLS)

Proteção verificada	Evidência
RLS ligada em todas as tabelas de public (37 iniciais e a partição events_default) por um laço sistemático; as 36 tabelas criadas depois ligam RLS na própria migration.	supabase/migrations/0002_rls.sql:130-136; 0004:250-252, 0040:461-465, 0073:33 etc.
Teste de integração dedicado à RLS, com lista de tabelas sensíveis.	tests/security/rls.test.ts
Toda função security_definer fixa search_path; as de pipeline e retenção são revogadas de public, anon e authenticated.	ex.: 0009_events_retention.sql:37-38, 0041:102
Funções auxiliares de papel (has_role, is_staff...) fora de anon; as políticas lidas por anon usam subconsulta, sem chamá-las.	0002_rls.sql:116-125, 0002_rls.sql:163-165
Regra de duas pessoas e escrita direta bloqueada por triggers no banco (publicação, regras, prompts, pesos, admin, push urgente).	0002:415-688, 0023:860-955, 0047_push_gate_fixes.sql:76-170
Dados pessoais isolados: reader_preferences só do dono; push_subscriptions sem colunas de chave para ninguém além do serviço (grant por coluna); reader_emails e newsletter_subscriptions sem política.	0020:1-35, 0040:466-479, 0012:24-25
audit_log é append-only (trigger imutável e revoke de update/delete).	0001_init.sql:267-268, 0002_rls.sql:383
Bucket media privado, servido por URL assinada curta; leitura pública de media_assets só approved e com a flag de reprodução.	0004_pipeline.sql:636-649, src/lib/media/serve.ts:46-47
Service role nunca recebe filtro controlado pelo usuário em rota pública: e-mail vem só de token HMAC; push exige hash do token de gestão.	src/lib/newsletter/token.ts:43-73, src/lib/push/api.ts:234-238

2. Permissão definida no navegador

Proteção verificada	Evidência
Wrapper único studioAction: valida a entrada com zod, checa a ação sem escopo, resolve o escopo do objeto com o cliente da sessão, recheia com escopo, respeita read_only e audita. Usado por quase todas as mutações do Estúdio.	src/lib/studio/action.ts:77-130
141 Server Actions percorridas: todas autenticam e checam o papel no servidor (ex.: fontes/actions.ts via context() → requireRole("source.manage"); notificações via requireAnyRole + checagem fina).	src/app/estudio/control/fontes/actions.ts:106-107, src/app/estudio/admin/notificacoes/actions.ts:66, 71, 174, 242, 266, 315
Os gates da UI têm equivalente no servidor: PromptVersions (rollback só admin ou editor-chefe), WeightsHistory, PendingApprovalsPanel, PushQueueTable, ArticleEditor/PublishDialog.	src/lib/studio/ai-prompts.ts:162-166, 215-219, recommendation.ts:118-127
Autoaprovação barrada no servidor e no banco.	src/lib/studio/approvals.ts:84-102, fontes/actions.ts:531-533
Rotas de cron e worker (6) exigem Bearer CRON_SECRET comparado com timingSafeEqual sobre SHA-256 e falham fechadas sem segredo.	src/lib/security/cron-auth.ts:11-17; src/app/api/jobs/drain/route.ts:13
Sessão da equipe com duração máxima: vencida, os papéis saem da sessão (falha fechada).	src/lib/auth/require-role.ts:31-36

3. IDOR (objeto por ID sem checar posse ou escopo)

Proteção verificada	Evidência
api/push/subscriptions/[id] (PATCH/DELETE) e rotate usam a service role, mas exigem o hash do Bearer do dono comparado em tempo constante: não há IDOR.	src/lib/push/api.ts:221-238, 286-288, src/lib/push/token.ts:17-21
Alertas e newsletter: o e-mail-alvo vem do token HMAC com finalidade e expiração, nunca do formulário; o update é restrito por owner_ref do token.	src/lib/db/writes.ts:223-238, src/lib/newsletter/token.ts:43-68
Perfil, migração e senha gravam só em reader.user.id, com o cliente da sessão sob RLS.	src/app/(public)/perfil/actions.ts, src/app/(public)/entrar/migrar/actions.ts
api/media/[id] público: cliente anon (RLS approved) mais nova checagem de status e da flag de reprodução.	src/lib/media/serve.ts:46-47
Correções e publicação forçada repetem can(section) por objeto; correção confere que a denúncia pertence à matéria.	src/lib/studio/forced-publish.ts:91-94, src/lib/studio/corrections.ts:34-43
Sem redirecionamento aberto: next passa por internalPath (decodifica até 3 camadas, recusa //, \ e caracteres de controle, exige a mesma origem).	src/lib/auth/safe-path.ts:11-30, src/app/auth/callback/route.ts:21

4. Chaves expostas (hardcode)

Proteção verificada	Evidência
Nenhum segredo real em arquivos rastreados nem no histórico git (213 commits); nenhum .env, .pem ou logs/*.jsonl já foi commitado.	git log --all -G (padrões acima); logs/.gitkeep é o único arquivo em logs/
pg_cron e pg_net leem o segredo do Vault; nada hardcoded nas migrations.	0004_pipeline.sql:305-320, 0053_agenda_collector.sql:50-54
Só NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY e NEXT_PUBLIC_VAPID_PUBLIC_KEY vão para o bundle; a service role vive em módulo server-only.	src/lib/db/client.ts:1
Fallbacks de desenvolvimento falham fechados em produção (token da newsletter e sal do rate limit devolvem null).	src/lib/newsletter/token.ts:17-21, src/lib/security/rate-limit.ts:27-38
Sem chave do OpenRouter, cai no provedor fake em vez de usar um valor padrão.	src/lib/ai/openrouter.ts:21
Workflows usam secrets.*, mascaram valores e declaram permissions: contents: read.	.github/workflows/ci.yml

5. Inputs sem tratamento (XSS e injeções de saída)

Proteção verificada	Evidência
Os seis dangerouslySetInnerHTML são seguros: sprite estático, script de tema constante com nonce e JSON-LD escapado (< vira \u003c).	src/lib/seo/jsonld.ts:153-155, src/app/layout.tsx:50, 65, src/components/ui/IconSprite.tsx:17
Nenhum innerHTML, eval, new Function ou renderizador de markdown no projeto.	grep em src/; package.json
CSP forte: nonce por requisição, strict-dynamic, object-src 'none', base-uri 'self', frame-ancestors 'none', HSTS e nosniff.	src/lib/security/headers.ts:24-56, src/proxy.ts:47-67
URLs externas validadas na entrada (só http/https; agenda e anúncios só https).	src/lib/db/queries/aggregated.ts:35-39, src/lib/agenda/submission.ts:97, src/lib/studio/admin-ops.ts:37-45
Clique em notificação só abre caminho interno ou a mesma origem (safeTarget).	src/sw/core.ts:127-130, 172-184

Proteção verificada	Evidência
SVG recusado em todos os caminhos de imagem; Content-Type sai do formato analisado, com nosniff.	src/lib/media/fetch-image.ts:84-86, src/lib/media/serve.ts:63-70
sanitizeExternalText aplicado em todo o pipeline de coleta e IA; e-mails ao leitor são texto puro.	src/lib/security/sanitize.ts; src/lib/db/writes.ts:167-192
Exportações CSV de logs e push protegidas contra fórmula.	src/lib/control/index.ts:191-196, src/lib/push/csv.ts:5-11

Pontos fracos (riscos centrais)

- Políticas de RLS antigas mais amplas que as views criadas depois para recortar o dado: `public_article_versions` e as colunas públicas de `articles` são contornáveis lendo a tabela diretamente pelo PostgREST (C1-01, C1-04).
- Funções `security_definer` liberadas a `authenticated` sem checar papel. Com cadastro público aberto, `authenticated` = qualquer pessoa (C1-03, C1-05, C1-07, C1-08).
- Identidade por e-mail sem prova de posse quando a confirmação de e-mail está desligada (`enable_confirmations = false`), afetando exportação e exclusão LGPD (C1-02).
- Escopo por editoria checado só no objeto de entrada, e não no conjunto afetado, em operação em lote com `service role` (C3-01).
- Higiene de segredos dependente do operador: `.gitignore` incompleto para `.env.prod`, `CRON_SECRET` sem validação de força e reaproveitado como chave HMAC, backup sem cifra em repositório privado (C4-01, C4-02, C4-03, C4-04).

Achados detalhados

1. Banco sem tranca (isolamento por RLS)

Severidade	Arquivo:linha	Descrição
MÉDIA	supabase/migrations/0002_rls.sql:164-165 supabase/migrations/0008_portal.sql:64-73 (view que deveria recortar)	C1-01 · Anônimos leem todas as versões (inclusive rascunhos pré-publicação) de matérias publicadas A política <code>article_versions_read_public</code> libera a anon a tabela inteira <code>article_versions</code> de qualquer matéria publicada ou atualizada: o snapshot completo (título, corpo, tags, SEO, <code>fieldOrigins</code>), <code>origin ('ai'/'human')</code> e <code>author_id</code> , incluindo versões anteriores à publicação. A view <code>public_article_versions</code> foi criada para expor só as versões pós-publicação, sem autor nem origem, mas a política da tabela anula esse recorte. Explorável com GET <code>/rest/v1/article_versions?select=*&article_id=eq.<id></code> e a anon key do bundle. <i>Condições:</i> Nenhuma: vale para toda matéria publicada.
MÉDIA	supabase/config.toml:225 supabase/migrations/0021_reader_email_gate.sql:112-144 supabase/migrations/0021_reader_email_gate.sql:47-63 supabase/migrations/0024_profiles_on_delete.sql:43-89	C1-02 · Sem confirmação de e-mail, quem cadastra o e-mail de outra pessoa exporta e apaga os dados dela <code>export_email_data()</code> (<code>security definer</code> , liberada a <code>authenticated</code>) devolve as inscrições na newsletter, os alertas (com alvos seguidos) e os metadados de e-mails ligados ao e-mail da conta, exigindo só <code>email_confirmed_at is not null</code> . Com <code>enable_confirmations = false</code> , o <code>GoTrue</code> marca o e-mail como confirmado no cadastro. O atacante cria conta com o e-mail da vítima (que assina newsletter ou alertas sem ter conta), chama <code>rpc('export_email_data')</code> e depois pede exclusão da conta: <code>purge_deleted_accounts</code> chama <code>purge_email_data</code> , que apaga newsletter, alertas e fila da vítima. <i>Condições:</i> Produção com "Confirm email" desligado, como no <code>config.toml</code> versionado. A vítima não pode ter conta com o mesmo e-mail.
MÉDIA	supabase/migrations/0041_push_admin.sql:138-153 supabase/migrations/0002_rls.sql:379-380	C1-03 · Qualquer conta grava entradas falsas e imutáveis no audit_log via push_audit <code>push_audit</code> é <code>security definer</code> , liberada a <code>authenticated</code> e só exige que a ação comece com <code>push..</code> Não checa papel, tamanho de <code>p_details</code> nem frequência. Uma conta de leitor (cadastro aberto) chama <code>rpc('push_audit', {p_action: 'push.approve', p_object: 'push:<id>', p_details: {...}})</code> e forja trilhas de aprovação de <code>push</code> . Como <code>audit_log</code> é imutável, as entradas não podem ser removidas, o que também permite inflar o armazenamento. Agravante: a política <code>audit_log_insert</code> deixa qualquer papel da equipe (inclusive leitura) inserir qualquer <code>action</code> , contornando a lista fechada de <code>studio_audit_actions()</code> . <i>Condições:</i> Só uma conta autenticada.
BAIXA	supabase/migrations/0002_rls.sql:150	C1-04 · Colunas internas de matérias publicadas legíveis por anônimos <code>articles_read_public</code> libera todas as colunas das matérias publicadas, sem <code>grant</code> por coluna: <code>author_id</code> , <code>assignee_id</code> , <code>agent_id</code> , <code>publish_mode</code> , <code>confidence_score</code> , <code>review_reason</code> , <code>ai_fallback</code> , <code>rules_version</code> , <code>field_origins</code> , <code>ai_summary_reviewed_by</code> , <code>due_at</code> , <code>short_reason</code> , <code>embedding</code> . Explorável com GET <code>/rest/v1/articles?select=*</code> . Expõe metadados editoriais e de IA que a UI pública deliberadamente esconde (CLAUDE.md §5.3). Não expõe dado de leitor. <i>Condições:</i> Nenhuma.

Severidade	Arquivo:linha	Descrição
BAIXA	supabase/migrations/0002_rls.sql:116-125 supabase/migrations/0008_portal.sql:27-32 supabase/migrations/0027_control_center.sql:235 supabase/migrations/0041_push_admin.sql:134-135	C1-05 · Oráculo de papéis: qualquer conta descobre quem é admin ou editor has_role, is_staff, has_any_role, can_edit_section, article_owner, article_section, control_can_* e push_can recebem um uid arbitrário e estão liberadas a authenticated. A view public_bylines entrega a anon o id e o nome de toda pessoa com papel. Uma conta comum chama rpc('has_role', {uid, role:'admin'}) e mapeia admins para phishing direcionado; article_owner e article_section revelam autor e editoria de rascunhos se o UUID for conhecido. <i>Condições:</i> Conta autenticada (cadastro aberto).
BAIXA	supabase/migrations/0002_rls.sql:337-340 supabase/migrations/0002_rls.sql:352	C1-06 · Eventos individuais e perfis de leitores lidos por papéis amplos da equipe events_read_metrics libera linhas individuais de events (com anon_id, user_id, session e termos buscados em props.query) a editor e leitura, que só precisam de agregados. profiles_read_staff deixa qualquer papel da equipe ler todos os perfis de leitores (nome, bairro, pedido de exclusão). É acesso maior que o necessário a dados pessoais (minimização, LGPD). <i>Condições:</i> Membro da equipe com papel leitura, editor, analista ou similar.
INFORMATIVA	supabase/migrations/0048_p5_gate_fixes.sql:32-61 supabase/migrations/0011_source_admin.sql:167-181 supabase/migrations/0041_push_admin.sql:102-103	C1-07 · Funções security definir sem checagem de papel, de baixo impacto approval_target_hash devolve o sha256 de regras, prompts e pesos (oráculo de conteúdo). lock_fast_lane_max permite que qualquer conta faça SELECT ... FOR UPDATE em app_settings (lock curto). push_settings_int lê chaves push.*. Todas estão liberadas a authenticated sem checar papel.
INFORMATIVA	supabase/migrations/0011_source_admin.sql:628-633 supabase/migrations/0048_p5_gate_fixes.sql:100 supabase/migrations/0027_control_center.sql:37-48	C1-08 · Revogação incompleta (from public, anon) e guarda auth.uid() is not null and ... consume_source_critical_approval e approval_assert_content foram revogadas só de public e anon, então seguem executáveis por authenticated (hoje inofensivas). Funções como control_guard_view, contingency_pause_cycle, prompt_rollback e rules_rollback deixam passar quando auth.uid() é nulo; estão seguras só porque anon foi revogado. É um padrão frágil para mudanças futuras.

Trecho C1-01 — supabase/migrations/0002_rls.sql:164-165

```
create policy article_versions_read_public on article_versions for select to anon, authenticated
using (exists (select 1 from articles a where a.id = article_id
and a.status in ('published', 'updated')));
```

Trecho C1-02 — supabase/config.toml:225

```
# supabase/config.toml:225
enable_confirmations = false
-- 0021:123-124
select lower(u.email) into e from auth.users u
where u.id = (select auth.uid()) and u.email_confirmed_at is not null;
```

Trecho C1-03 — supabase/migrations/0041_push_admin.sql:138-153

```
create or replace function public.push_audit(p_action text, p_object text, p_details jsonb ...)
... security definer ...
if p_action not like 'push.%' then raise exception ...
insert into audit_log (actor, action, object_ref, details) values (...);
...
grant execute on function public.push_audit(text, text, jsonb) to authenticated, service_role;
```

Trecho C1-04 — supabase/migrations/0002_rls.sql:150

```
create policy articles_read_public on articles for select to anon, authenticated
using (status in ('published', 'updated'));
```

Trecho C1-05 — supabase/migrations/0002_rls.sql:116-125

```
grant execute on function public.has_role(uuid, app_role, text), public.is_staff(uuid),
public.has_any_role(uuid, app_role[]), public.can_edit_section(uuid, text), ...
to authenticated, service_role;
```

Trecho C1-06 — supabase/migrations/0002_rls.sql:337-340

```
create policy events_read_metrics on events for select to authenticated
using (has_any_role((select auth.uid()), '{admin,editor_chefe,editor,operador_ia,analista,leitura}'));
```

2. Permissão definida no navegador

Severidade	Arquivo:linha	Descrição
BAIXA	src/app/(public)/perfil/actions.ts:20-23 src/components/editorial/AccountForms.tsx:53	C2-01 · Validação do bairro do perfil confia em valor-base enviado pelo navegador A ação aceita o bairro se ele for igual a <code>current_neighborhood</code> , campo oculto vindo do próprio formulário. Enviar <code>neighborhood=X&current_neighborhood=X</code> grava qualquer texto, sem limite de tamanho, em <code>profiles.neighborhood</code> . O alcance é só o próprio perfil, e <code>profiles_update_self</code> (0002:354) já permite o mesmo pelo PostgREST: a validação do servidor é ilusória e o banco não tem check de tamanho ou lista. <i>Condições:</i> Leitor autenticado, só no próprio perfil.
INFORMATIVA	src/lib/push/api.ts:294-306	C2-02 · Recibo de push: consentimento lido de cookie do cliente e sendId sem vínculo O consentimento vem do cookie <code>cn_consent</code> , que o próprio cliente define, e o <code>sendId</code> não é ligado a nenhuma inscrição. Com o Origin certo, qualquer pessoa infla de <code>delivered/clicked</code> e o CTR de qualquer envio. O limite de 120 recibos por hora por IP atenua.
INFORMATIVA	src/app/estudio/actions.ts:258-271	C2-03 · Push urgente em publishAction sem checagem na própria Server Action A UI só oferece o urgente com <code>canRequestUrgent</code> , mas a ação chama <code>requestUrgentPush</code> sem rechecar. Não é explorável: o <code>trigger_guard_push_sends barra</code> (0047_push_gate_fixes.sql:115-116). Falta uma camada de defesa, mas não há falha.

Trecho C2-01 — src/app/(public)/perfil/actions.ts:20-23

```
const current = String(form.get("current_neighborhood") ?? "").trim();
const allowed = hood === "" || hood === current || NEIGHBORHOODS.some((n) => n.name === hood);
```

3. IDOR (objeto por ID sem checar posse ou escopo)

Severidade	Arquivo:linha	Descrição
MÉDIA	src/lib/studio/media.ts:119-137 src/app/estudio/actions.ts:367-373 src/lib/db/pipeline-store.ts:1060-1068 src/lib/auth/permissions.ts:73	C3-01 · Editor de uma editoria remove imagens de todas as editorias ("remover todas da fonte") <code>takedownImage</code> checa <code>media.approve</code> só no escopo da imagem passada em <code>id</code> (<code>mediaScope</code>). Com <code>allFromSource: true</code> , a remoção vai para todas as reproduções do mesmo <code>source_id</code> , sem filtro de editoria, e roda com a <code>service role</code> , que ignora a RLS e apaga a cópia no Storage. Um editor com <code>sections=["esportes"]</code> escolhe uma reprodução da própria editoria e remove, de forma irreversível, as imagens daquele veículo em matérias de todas as outras editorias. <i>Condições:</i> Papel editor restrito a editoria e uma reprodução da fonte ligada a uma matéria da editoria dele. Para revisor e editor-chefe (escopo a <code>ll</code>) não há escalada.

Severidade	Arquivo:linha	Descrição
BAIXA	src/app/api/estudio/midia/[id]/route.ts:21-22 src/app/api/estudio/midia/[id]/route.ts:44-50 supabase/migrations/0002_rls.sql:190	C3-02 · Prévia de mídia do Estúdio serve qualquer imagem (pendente ou bloqueada) a qualquer papel A rota exige só que a sessão tenha algum papel (<code>roles.length > 0</code>), enquanto as telas exigem <code>media.approve</code> ou <code>article.edit</code> . Ela força <code>status: "approved"</code> e ignora a flag <code>image_reproduction_enabled</code> . Quem tem leitura, analista, moderador ou operador_ia lista os ids pelo PostgreSQL (<code>media_assets_read_staff</code>) e baixa os bytes de imagens bloqueadas por motivo legal que ainda estejam no Storage. O jornalista vê imagens que não são das matérias dele. <i>Condições:</i> Conta com qualquer papel da equipe.
INFORMATIVA	src/lib/reports/report.ts:33 src/lib/db/writes.ts:241-256	C3-03 · Denúncia aceita qualquer contentRef com formato de UUID O código não confere se o conteúdo existe ou é público antes de gravar com a service role. Só polui a fila de denúncias (limite de 5 por hora por IP) e não escala: <code>openCorrection</code> confere a matéria.

Trecho C3-01 — src/lib/studio/media.ts:119-137

```
export const takedownImage = studioAction(
  "media.approve",
  (i: TakedownInput, ctx) => mediaScope(ctx, i.id), // escopo de UMA imagem
  async (i, ctx) => { ...
    const service = createServiceClient();
    const target = i.allFromSource && m.source_id ? { sourceId: m.source_id } : { mediaId: i.id };
  }
```

Trecho C3-02 — src/app/api/estudio/midia/[id]/route.ts:21-22

```
const session = await getSession();
if (!session || session.roles.length === 0) return notFound();
...
status: "approved",
...
reproductionEnabled: async () => true,
```

4. Chaves expostas (hardcode)

Severidade	Arquivo:linha	Descrição
MÉDIA	.gitignore:3-4 scripts/release-backlog.mjs:11	C4-01 · .gitignore não cobre .env.prod e um script manda criá-lo com a service role de produção Só <code>.env</code> e <code>.env*.local</code> são ignorados; <code>git check-ignore</code> confirma que <code>.env.prod</code> , <code>.env.production</code> e <code>.env.development</code> não estão ignorados. O fluxo documentado (<code>node --env-file=.env.prod ...</code>) leva o operador a criar <code>.env.prod</code> com <code>SUPABASE_SERVICE_ROLE_KEY</code> de produção. Um <code>git add -A</code> (o agente autônomo tem <code>Bash(git:*)</code> liberado) commitaria a chave, que ignora toda a RLS. <i>Condições:</i> O arquivo precisa ser criado e commitado. Hoje não existe nem no disco nem no histórico.
BAIXA	src/lib/security/cron-auth.ts:11-17 .env.example:8	C4-02 · CRON_SECRET sem validação de força nem checagem de placeholder na inicialização <code>isCronAuthorized</code> só recusa segredo vazio. Não há módulo de validação de ambiente nem <code>instrumentation.ts</code> . Se <code>.env.example</code> for copiado sem troca, o valor público substituir <code>-por-32-caracteres-aleatorios</code> autoriza as 6 rotas de cron (<code>ingest</code> e <code>jobs</code>) e vira a chave HMAC dos links da newsletter. <i>Condições:</i> Erro de configuração do operador.

Severidade	Arquivo:linha	Descrição
BAIXA	.github/workflows/backup.yml:35 .github/workflows/backup.yml:63-68	C4-04 · Backup de produção (com auth.users) sai sem cifra em repositório privado O dump inclui o schema auth (hashes de senha, e-mails). A cifra GPG só é exigida quando o repositório é público; em repositório privado, o artifact fica 7 dias sem cifra, baixável por qualquer pessoa com leitura no repositório (colaboradores, tokens, apps). <i>Condições:</i> Repositório privado sem o secret BACKUP_PASSPHRASE.
INFORMATIVA	src/lib/newsletter/token.ts:17-18 src/lib/security/rate-limit.ts:27-28	C4-03 · CRON_SECRET reaproveitado como chave HMAC da newsletter e sal do rate limit Sem NEWSLETTER_TOKEN_SECRET ou RATE_LIMIT_SALT, os dois módulos usam o CRON_SECRET. Um vazamento do segredo de cron também permite forjar links de preferências de newsletter e alertas para qualquer e-mail. Os fallbacks de desenvolvimento falham fechados em produção (ponto forte).
INFORMATIVA	scripts/local-stack/env.sh:19 scripts/local-stack/start.sh:99 supabase/seed.sql:64 README.md:80,103	C4-05 · Credenciais locais e de teste conhecidas, e ref do projeto de produção na documentação JWT_SECRET padrão do Supabase local, CRON_SECRET local, senha citynews-local-123 dos usuários de seed e ci-secret-... no CI: todos só para ambiente local ou CI (o seed não roda em produção). O README e .planning/ citam o ref vmvirmemxftxfdmivuu, já público pela NEXT_PUBLIC_SUPABASE_URL. Sem ação necessária além de garantir que esses valores nunca vão para produção.

Trecho C4-01 — .gitignore:3-4

```
# .gitignore:3-4
.env*.local
.env
# scripts/release-backlog.mjs:11
// node --env-file=.env.prod scripts/release-backlog.mjs --apply --confirm-host=<projeto>.supabase.co
```

Trecho C4-02 — src/lib/security/cron-auth.ts:11-17

```
export function isCronAuthorized(header: string | null, secret: string | undefined): boolean {
  if (!secret) return false; // aceita qualquer valor não vazio, inclusive o placeholder
```

Trecho C4-04 — .github/workflows/backup.yml:35

```
elif [ "$REPO_PRIVATE" != "true" ] && [ -z "${BACKUP_PASSPHRASE:-}" ]; then # só bloqueia se público
...
if [ -n "${BACKUP_PASSPHRASE:-}" ]; then gpg ... --symmetric ... # cifra opcional
```

5. Inputs sem tratamento (XSS e injeções de saída)

Severidade	Arquivo:linha	Descrição
BAIXA	src/lib/ics.ts:42-48 src/lib/ics.ts:94-97 src/app/api/ics/[slug]/route.ts:20-28	C5-01 · Injeção em arquivo ICS; ; não é escapado e CR sozinho não é tratado Em JS, "\";" é só ";", então o ponto e vírgula não é escapado. /\r?\n/ deixa passar um \r sozinho, que parsers tolerantes tratam como quebra de linha: um título ou descrição com \rATTACH:... ou \rEND:VEVENT injeta propriedades ou eventos no .ics baixado. O texto coletado passa por sanitizeExternalText (normaliza CR), mas a sugestão pública de evento (src/lib/agenda/submission.ts) não remove caracteres de controle. <i>Condições:</i> Sugestão de evento com \r enviada por POST manual e aprovada pela equipe. Não é XSS no portal.
BAIXA	src/components/studio/sources/SourceAuditTable.tsx:79-96	C5-02 · Injeção de fórmula no CSV do histórico de fontes csvCell só dobra aspas; r.actor.name e r.reason começando com =, +, - ou @ viram fórmula (ex.: =HYPERLINK(...)) no Excel de quem exportar. As outras exportações usam toCsv, que já protege. <i>Condições:</i> Membro da equipe mal-intencionado (nome de exibição ou motivo) e a vítima abrir o CSV numa planilha.

Trecho C5-01 — src/lib/ics.ts:42-48

```
function escape(text: string): string {
  return text
    .replace(/\\/g, "\\")
    .replace(/;/g, "\\;") // "\\;" === ";": não escapa
    .replace(/,/g, "\\,")
    .replace(/\r?\n/g, "\\n"); // CR sozinho passa
}
```

Trecho C5-02 — src/components/studio/sources/SourceAuditTable.tsx:79-96

```
const csvCell = (s: string) => `${s.replace(/"/g, '"')}";`
...
r.actor.name, ..., r.reason ?? "", ... ].map(csvCell)
```

Recomendações priorizadas

Prioridade	Ação	Achados
P1	Remover a política <code>article_versions_read_public</code> e servir o histórico público só pela <code>view public_article_versions</code> .	C1-01
P1	Ligar "Confirm email" no Supabase de produção (e em <code>config.toml</code>). Até lá, recusar <code>export_email_data</code> e o expurgo por e-mail para contas cujo e-mail não foi confirmado por link.	C1-02
P1	<code>push_audit</code> : exigir <code>push_can(auth.uid(), ...)</code> , limitar o tamanho de <code>p_details</code> e aplicar <code>rate limit</code> . Restringir <code>audit_log_insert</code> à lista de <code>studio_audit_actions()</code> ou removê-la.	C1-03
P1	<code>takedownImage</code> com <code>allFromSource</code> : exigir <code>media.approve</code> com escopo <code>all</code> , ou filtrar o lote pelas editorias do usuário.	C3-01
P1	<code>.gitignore</code> : trocar por <code>.env*</code> com <code>!.env.example</code> .	C4-01
P2	Grant de <code>SELECT</code> por coluna em <code>articles</code> para <code>anon</code> (ou servir o portal por <code>view</code>) e revogar as funções de papel com <code>uid</code> arbitrário de <code>authenticated</code> , trocando por <code>wrappers</code> baseados em <code>auth.uid()</code> .	C1-04, C1-05
P2	Rota de prévia de mídia do Estúdio: exigir <code>media.approve</code> ou <code>article.edit</code> com escopo do objeto e respeitar <code>blocked</code> .	C3-02
P2	Validação de ambiente na inicialização (<code>instrumentation.ts</code>): <code>CRON_SECRET</code> com 32 caracteres ou mais e nunca o placeholder. Segredos separados para <code>newsletter</code> e <code>rate limit</code> .	C4-02, C4-03
P2	Backup: exigir <code>BACKUP_PASSPHRASE</code> sempre, inclusive em repositório privado.	C4-04
P3	Escape correto no ICS (<code>\;</code> , <code>\r\n</code> ou <code>\r</code> e remoção de controles) e proteção contra fórmula no CSV de fontes.	C5-01, C5-02
P3	Dar a editor e leitura agregados em vez de <code>events crus</code> ; rever <code>profiles_read_staff</code> .	C1-06
P3	Bairro: validar só pela lista (sem campo-base do cliente) e adicionar check no banco.	C2-01
P3	Endurecimento: padronizar <code>revoke ... from public, anon, authenticated</code> , falhar quando <code>auth.uid()</code> for nulo fora da <code>service role</code> , rechecar <code>canRequestUrgent</code> na ação, validar a existência de <code>contentRef</code> e vincular o recibo de <code>push</code> à inscrição.	C1-07, C1-08, C2-02, C2-03, C3-03

ISSUES PARA O GITHUB

Cada bloco abaixo é o texto completo de uma issue em Markdown, pronto para copiar e colar.

```
--- ISSUE 1 ---
## [Segurança] Anônimos leem todas as versões de matérias publicadas, inclusive rascunhos pré-publicação

**Labels sugeridas:** `security`, `severity:média`

### Descrição
A política RLS `article_versions_read_public` libera a `anon` e a `authenticated` todas as linhas de `article_versions` de matérias publicadas ou atualizadas, com o `snapshot` completo, `origin` (ai/human) e `author_id`. A view `public_article_versions` foi criada para expor só as versões pós-publicação e sem esses campos, mas o PostgREST permite ler a tabela diretamente com a anon key pública: `GET /rest/v1/article_versions?select=*%&article_id=eq.<id>`.

### Evidência
- **C1-01** — `supabase/migrations/0002_rls.sql:164-165`, `supabase/migrations/0008_portal.sql:64-73` (view que deveria recortar)
  ```
 ...
 create policy article_versions_read_public on article_versions for select to anon, authenticated
 using (exists (select 1 from articles a where a.id = article_id
 and a.status in ('published', 'updated')));
 ...
  ```

### Impacto
Vaza texto de rascunhos que nunca foram publicados (inclusive trechos removidos na revisão), a origem IA ou humana de cada campo e o UUID dos autores. Contraria a regra de produto §5.3 (nenhuma tela pública expõe origem IA).

### Sugestão de correção
Nova migration com `drop policy article_versions_read_public on article_versions;`. Se algum caminho público ler a tabela, trocar pela view `public_article_versions`. Adicionar o caso a `tests/security/rls.test.ts`.

### Critérios de aceite
- [ ] `select` anônimo em `article_versions` devolve 0 linhas
- [ ] `public_article_versions` continua devolvendo só versões pós-publicação
- [ ] Teste de RLS cobre o caso
- [ ] `pnpm verify` verde
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
## [Segurança] Sem confirmação de e-mail, conta com e-mail alheio exporta e apaga dados da vítima

**Labels sugeridas:** `security`, `severity:média`

### Descrição
`export_email_data()` e o expurgo de conta (`purge_deleted_accounts` → `purge_email_data`) confiam em `auth.users.email_confirmed_at`. Com `enable_confirmations = false`, o GoTrue confirma o e-mail no cadastro, então qualquer pessoa cria conta com o e-mail de um assinante e passa a ser tratada como dona dos dados dele.

### Evidência
- **C1-02** — `supabase/config.toml:225`, `supabase/migrations/0021_reader_email_gate.sql:112-144`, `supabase/migrations/0021_reader_email_gate.sql:47-63`, `supabase/migrations/0024_profiles_on_delete.sql:43-89`
  ```
 ...
 # supabase/config.toml:225
 enable_confirmations = false
 -- 0021:123-124
 select lower(u.email) into e from auth.users u
 where u.id = (select auth.uid()) and u.email_confirmed_at is not null;
 ...
  ```

### Impacto
Leitura (inscrições, alertas e assuntos seguidos) e exclusão de dados pessoais de terceiros, com efeito sobre direitos LGPD de acesso e eliminação.

### Sugestão de correção
Ligar "Confirm email" no projeto de produção e em `supabase/config.toml`. Como defesa adicional, registrar a confirmação por link próprio (ex.: `reader_emails` confirmado) e exigir isso em `export_email_data` e `purge_email_data`.

### Critérios de aceite
- [ ] Cadastro novo não tem `email_confirmed_at` até clicar no link
- [ ] `export_email_data` devolve null para conta sem confirmação por link
- [ ] Expurgo não apaga dados por e-mail de conta não confirmada
- [ ] Teste de integração cobre o cenário
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
## [Segurança] push_audit permite que qualquer conta grave entradas falsas e imutáveis no audit_log

**Labels sugeridas:** `security`, `severity:média`

### Descrição
`public.push_audit` é `security definer`, liberada a `authenticated` (cadastro aberto) e só exige `p_action`
like `push.%`. Não checa papel, tamanho nem frequência. Além disso, a política `audit_log_insert` deixa
qualquer papel da equipe inserir qualquer `action` diretamente.

### Evidência
- **C1-03** — `supabase/migrations/0041_push_admin.sql:138-153`, `supabase/migrations/0002_rls.sql:379-380`
  ...
  create or replace function public.push_audit(p_action text, p_object text, p_details jsonb ...)
  ... security definer ...
    if p_action not like 'push.%' then raise exception ...
    insert into audit_log (actor, action, object_ref, details) values (...);
  ...
  grant execute on function public.push_audit(text, text, jsonb) to authenticated, service_role;
  ...

### Impacto
Forja da trilha de auditoria (ex.: `push.approve` falso), que não pode ser apagada por causa da imutabilidade,
e crescimento ilimitado da tabela.

### Sugestão de correção
Em `push_audit`: exigir `push_can(auth.uid(), ...)` ou `is_staff`, limitar `p_details` a cerca de 1 KB e
aplicar `hit_rate_limit`, como já faz `studio_audit` (0025). Restringir `audit_log_insert` a `action = any(
studio_audit_actions())` ou removê-la, centralizando a gravação em funções.

### Critérios de aceite
- [ ] Leitor sem papel recebe 42501 ao chamar `push_audit`
- [ ] `p_details` acima do limite é recusado
- [ ] Insert direto com action fora da lista falha
- [ ] Testes de RLS e RPC adicionados
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
## [Segurança] Editor de editoria remove imagens de outras editorias via "remover todas da fonte"

**Labels sugeridas:** `security`, `severity:média`

### Descrição
`takedownImage` (`src/lib/studio/media.ts:119-137`) valida `media.approve` só no escopo da imagem `id`. Com
`allFromSource: true`, a remoção atinge todas as reproduções do mesmo `source_id`, com a service role, sem
filtrar por editoria.

### Evidência
- **C3-01** — `src/lib/studio/media.ts:119-137`, `src/app/estudio/actions.ts:367-373`, `src/lib/db/pipeline-
store.ts:1060-1068`, `src/lib/auth/permissions.ts:73`
  ...
  export const takedownImage = studioAction(
    "media.approve",
    (i: TakedownInput, ctx) => mediaScope(ctx, i.id), // escopo de UMA imagem
    async (i, ctx) => { ...
      const service = createServiceClient();
      const target = i.allFromSource && m.source_id ? { sourceId: m.source_id } : { mediaId: i.id };
    }
  );

### Impacto
Um editor restrito a uma editoria apaga, de forma irreversível (Storage), as imagens de reprodução daquele
veículo em matérias de todas as editorias.

### Sugestão de correção
Para `allFromSource`, exigir `can(session, "media.approve", { section: "*" })` (escopo `all`) ou filtrar as
reproduções às matérias das editorias do usuário. Esconder o checkbox para quem não tem escopo total.

### Critérios de aceite
- [ ] Editor com `sections=[x]` recebe `forbidden` ao usar `allFromSource`
- [ ] Editor-chefe e revisor continuam podendo
- [ ] Teste unitário de `takedownImage` cobre os dois papéis
--- FIM ISSUE 4 ---
```

```

--- ISSUE 5 ---
## [Segurança] .gitignore não cobre .env.prod / .env.production (risco de commitar service role)

**Labels sugeridas:** `security`, `severity:média`

### Descrição
O `.gitignore` só ignora `.env` e `.env*.local`. `scripts/release-backlog.mjs` documenta o uso de `.env.prod` com credenciais de produção, que não seria ignorado.

### Evidência
- **C4-01** — `.gitignore:3-4`, `scripts/release-backlog.mjs:11`
  ```
 # .gitignore:3-4
 .env*.local
 .env
 # scripts/release-backlog.mjs:11
 // node --env-file=.env.prod scripts/release-backlog.mjs --apply --confirm-host=<projeto>.supabase.co
  ```

### Impacto
Um `git add -A` (manual ou do agente autônomo) publicaria a `SUPABASE_SERVICE_ROLE_KEY` de produção, que ignora toda a RLS.

### Sugestão de correção
Trocar as linhas por `.env*` e `!.env.example`. Opcional: hook de pre-commit ou secret scanning (gitleaks) no CI.

### Critérios de aceite
- [ ] `git check-ignore .env.prod .env.production .env.development` retorna os três
- [ ] `.env.example` continua rastreado
- [ ] CI roda secret scanning (opcional)
--- FIM ISSUE 5 ---

```

```

--- ISSUE 6 ---
## [Segurança] Metadados internos de matérias e papéis da equipe expostos via PostgREST

**Labels sugeridas:** `security`, `severity:baixa`

### Descrição
`articles_read_public` libera todas as colunas de matérias publicadas (autor, `confidence_score`, `ai_fallback`, `field_origins`, `embedding`...). Funções como `has_role(uid, role)` recebem `uid` arbitrário e estão liberadas a `authenticated`; com os ids de `public_bylines`, qualquer conta mapeia quem é admin.

### Evidência
- **C1-04** — `supabase/migrations/0002_rls.sql:150`
  ```
 create policy articles_read_public on articles for select to anon, authenticated
 using (status in ('published', 'updated'));
  ```
- **C1-05** — `supabase/migrations/0002_rls.sql:116-125`, `supabase/migrations/0008_portal.sql:27-32`, `supabase/migrations/0027_control_center.sql:235`, `supabase/migrations/0041_push_admin.sql:134-135`
  ```
 grant execute on function public.has_role(uuid, app_role, text), public.is_staff(uuid),
 public.has_any_role(uuid, app_role[]), public.can_edit_section(uuid, text), ...
 to authenticated, service_role;
  ```

### Impacto
Vazamento de metadados editoriais e de IA e enumeração de contas privilegiadas para phishing direcionado.

### Sugestão de correção
`revoke select on articles from anon` + `grant select (colunas públicas) on articles to anon`, ou servir o portal por view. Revogar as funções de papel de `authenticated` e criar wrappers `my_*( )` baseados em `auth.uid()`; RLS continua usando as originais (as políticas rodam como dono).

### Critérios de aceite
- [ ] `GET /rest/v1/articles?select=confidence_score` anônimo falha
- [ ] `rpc('has_role')` de leitor falha
- [ ] Portal e Estúdio funcionam (e2e verde)
--- FIM ISSUE 6 ---

```

```
--- ISSUE 7 ---
## [Segurança] Papéis amplos da equipe leem eventos individuais e perfis de leitores

**Labels sugeridas:** `security`, `severity:baixa`

### Descrição
`events_read_metrics` libera linhas individuais de eventos (anon_id, user_id, buscas) a `editor` e `leitura`;
`profiles_read_staff` libera todos os perfis a qualquer papel.

### Evidência
- **C1-06** — `supabase/migrations/0002_rls.sql:337-340`, `supabase/migrations/0002_rls.sql:352`
  ```
 create policy events_read_metrics on events for select to authenticated
 using (has_any_role((select auth.uid()), '{admin,editor_chefe,editor,operador_ia,analista,leitura}'));
  ```

### Impacto
Acesso a dados pessoais além do necessário (minimização, LGPD).

### Sugestão de correção
Restringir `events` a admin e analista e expor agregados (views ou RPC) aos demais. Limitar
`profiles_read_staff` a papéis que moderam ou atendem leitores.

### Critérios de aceite
- [ ] Papel `leitura` não lê linhas de `events`
- [ ] Painéis de métricas continuam funcionando com agregados
- [ ] Teste de RLS atualizado
--- FIM ISSUE 7 ---
```

```
--- ISSUE 8 ---
## [Segurança] Prévia de mídia do Estúdio serve imagens pendentes ou bloqueadas a qualquer papel

**Labels sugeridas:** `security`, `severity:baixa`

### Descrição
`GET /api/estudio/midia/[id]` exige só `roles.length > 0`, força `status: approved` e ignora
`image_reproduction_enabled`, enquanto as telas exigem `media.approve` ou `article.edit`.

### Evidência
- **C3-02** — `src/app/api/estudio/midia/[id]/route.ts:21-22`, `src/app/api/estudio/midia/[id]/route.ts:44-50`,
  `supabase/migrations/0002_rls.sql:190`
  ```
 const session = await getSession();
 if (!session || session.roles.length === 0) return notFound();
 ...
 status: "approved",
 ...
 reproductionEnabled: async () => true,
  ```

### Impacto
Papéis como `leitura` ou `analista` baixam imagens bloqueadas (inclusive por motivo legal) e de matérias fora
do seu escopo.

### Sugestão de correção
Exigir `canAccess(session, "media.approve")` ou `article.edit` com escopo da matéria ligada; para imagens
`blocked`, só `media.approve`.

### Critérios de aceite
- [ ] Papel `leitura` recebe 404
- [ ] Editor de outra editoria recebe 404
- [ ] Teste da rota cobre os papéis
--- FIM ISSUE 8 ---
```

```
--- ISSUE 9 ---
## [Segurança] Validação do bairro do perfil depende de campo oculto do cliente

**Labels sugeridas:** `security`, `severity:baixa`

### Descrição
`perfil/actions.ts` aceita qualquer `neighborhood` igual a `current_neighborhood`, que vem do próprio formulário, e o banco não restringe a coluna.

### Evidência
- **C2-01** — `src/app/(public)/perfil/actions.ts:20-23`, `src/components/editorial/AccountForms.tsx:53`
  ```
 const current = String(form.get("current_neighborhood") ?? "").trim();
 const allowed = hood === "" || hood === current || NEIGHBORHOODS.some((n) => n.name === hood);
  ```

### Impacto
O leitor grava texto arbitrário e longo no próprio perfil (dado sujo, possível abuso em telas da equipe que listam perfis).

### Sugestão de correção
Ler o bairro atual do banco (não do form) ou validar só pela lista `NEIGHBORHOODS`. Adicionar `check (char_length(neighborhood) <= 80)` em `profiles`.

### Critérios de aceite
- [ ] POST com bairro fora da lista é recusado
- [ ] `check` de tamanho no banco
- [ ] Teste unitário da ação
--- FIM ISSUE 9 ---
```

```
--- ISSUE 10 ---
## [Segurança] Validar CRON_SECRET na inicialização e separar segredos derivados

**Labels sugeridas:** `security`, `severity:baixa`

### Descrição
`isCronAuthorized` aceita qualquer segredo não vazio, inclusive o placeholder de `.env.example`. O mesmo segredo vira chave HMAC da newsletter e sal do rate limit quando as variáveis dedicadas faltam.

### Evidência
- **C4-02** — `src/lib/security/cron-auth.ts:11-17`, `.env.example:8`
  ```
 export function isCronAuthorized(header: string | null, secret: string | undefined): boolean {
 if (!secret) return false; // aceita qualquer valor não vazio, inclusive o placeholder
 }
- **C4-03** — `src/lib/newsletter/token.ts:17-18`, `src/lib/security/rate-limit.ts:27-28`

Impacto
Má configuração libera as rotas de cron, e um único vazamento compromete cron, links de newsletter e alertas.

Sugestão de correção
Criar `src/instrumentation.ts` (ou `src/lib/env.ts`) que, em produção, rejeite `CRON_SECRET` com menos de 32 caracteres ou contendo `substituir`. Exigir `NEWSLETTER_TOKEN_SECRET` e `RATE_LIMIT_SALT` próprios em produção.

Critérios de aceite
- [] Boot em produção falha com o placeholder
- [] `isCronAuthorized` recusa segredo curto
- [] Newsletter e rate limit não usam mais o fallback para `CRON_SECRET` em produção
- [] Testes unitários
--- FIM ISSUE 10 ---
```

```

--- ISSUE 11 ---
[Segurança] Backup diário com auth.users sai sem cifra em repositório privado

Labels sugeridas: `security`, `severity:baixa`

Descrição
`.github/workflows/backup.yml` só exige `BACKUP_PASSPHRASE` em repositório público; em privado, o dump (schemas public, auth e storage) fica 7 dias como artifact sem cifra.

Evidência
- **C4-04** — `.github/workflows/backup.yml:35`, `.github/workflows/backup.yml:63-68`
  ```
  elif [ "$REPO_PRIVATE" != "true" ] && [ -z "${BACKUP_PASSPHRASE:-}" ]; then # só bloqueia se público
  ...
  if [ -n "${BACKUP_PASSPHRASE:-}" ]; then gpg ... --symmetric ... # cifra opcional
  ...
  ```

Impacto
Qualquer pessoa ou token com leitura no repositório baixa e-mails e hashes de senha de toda a base.

Sugestão de correção
Pular o backup (com aviso) sempre que faltar `BACKUP_PASSPHRASE` e cifrar sempre.

Critérios de aceite
- [] Sem passphrase, o job não publica artifact
- [] Artifact publicado é sempre `.gpg`
--- FIM ISSUE 11 ---

```

```

--- ISSUE 12 ---
[Segurança] Injeção em ICS e em CSV do histórico de fontes

Labels sugeridas: `security`, `severity:baixa`

Descrição
`src/lib/ics.ts` não escapa `;` (`"\\"` vira `\"` em JS) nem CR sozinho, permitindo injetar propriedades ou eventos no `ics`. `SourceAuditTable.tsx` monta o CSV sem neutralizar fórmulas (`=`, `+`, `-`, `@`).

Evidência
- **C5-01** — `src/lib/ics.ts:42-48`, `src/lib/ics.ts:94-97`, `src/app/api/ics/[slug]/route.ts:20-28`
  ```
  function escape(text: string): string {
    return text
      .replace(/\\/g, "\\")
      .replace(/;/g, "\\;") // ";" == ";": não escapa
      .replace(/,/g, "\\,")
      .replace(/\r?\n/g, "\\n"); // CR sozinho passa
  }
  ...
- **C5-02** — `src/components/studio/sources/SourceAuditTable.tsx:79-96`
  ```
 const csvCell = (s: string) => `${s.replace(/"/g, '"')}";
 ...
 r.actor.name, ..., r.reason ?? "", ...].map(csvCell)
 ...
  ```

### Impacto
Arquivo de calendário manipulado para quem baixa o evento; execução de fórmula na planilha de quem exporta o histórico.

### Sugestão de correção
ICS: `.replace(/;/g, "\\;")`, `.replace(/\r\n|\r|\n/g, "\\n")` e remoção de caracteres de controle (também em `agenda/submission.ts`). CSV: reutilizar o `toCsv` protegido de `src/lib/control`.

### Critérios de aceite
- [ ] Teste: `a;b\rX:1` sai como `a;b\nX:1`
- [ ] Teste: célula `=1+1` sai prefixada com ``
- [ ] `pnpm verify` verde
--- FIM ISSUE 12 ---

```

```
--- ISSUE 13 ---
## [Segurança] Endurecimento: grants de funções, guardas de uid nulo e validações de defesa em profundidade

**Labels sugeridas:** `security`, `severity:informativa`

### Descrição
Itens sem exploração relevante hoje que tornam a base frágil a mudanças: funções `security definer` sem
checagem de papel (`approval_target_hash`, `lock_fast_lane_max`, `push_settings_int`); revokes só `from public,
anon`; guardas `auth.uid() is not null and ...`; `publishAction` sem recheçar `canRequestUrgent`; recibo de
push sem vínculo com a inscrição; denúncia sem checar a existência do conteúdo.

### Evidência
- **C1-07** - `supabase/migrations/0048_p5_gate_fixes.sql:32-61`, `supabase/migrations/0011_source_admin.sql:
167-181`, `supabase/migrations/0041_push_admin.sql:102-103`
- **C1-08** - `supabase/migrations/0011_source_admin.sql:628-633`, `supabase/migrations/0048_p5_gate_fixes.sql:
100`, `supabase/migrations/0027_control_center.sql:37-48`
- **C2-02** - `src/lib/push/api.ts:294-306`
- **C2-03** - `src/app/estudio/actions.ts:258-271`
- **C3-03** - `src/lib/reports/report.ts:33`, `src/lib/db/writes.ts:241-256`

### Impacto
Baixo hoje, mas uma mudança futura (ex.: liberar anon numa função) vira falha real sem que nenhum teste acuse.

### Sugestão de correção
Padronizar `revoke execute ... from public, anon, authenticated` + `grant` explícito; nas funções só de serviço,
falhar se `auth.uid() is null and auth.role() <> 'service_role'`; checar papel nas três funções citadas;
recheçar `canRequestUrgent` na ação; validar `contentRef` com `findPublicArticleId`; assinar o `sendId` no
payload do push. Adicionar teste que liste funções `security definer` executáveis por authenticated e compare
com uma allowlist.

### Critérios de aceite
- [ ] Teste de allowlist de funções executáveis por `authenticated`
- [ ] Guardas de uid nulo revisadas
- [ ] Itens de defesa em profundidade aplicados
--- FIM ISSUE 13 ---
```